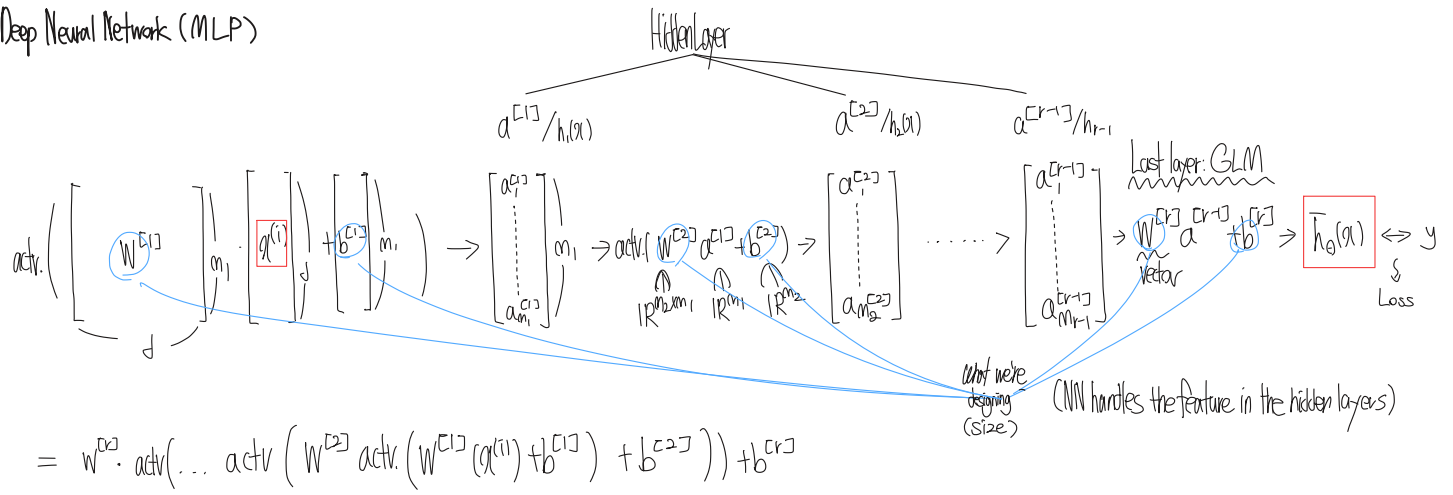


Deep Neural Network (MLP)

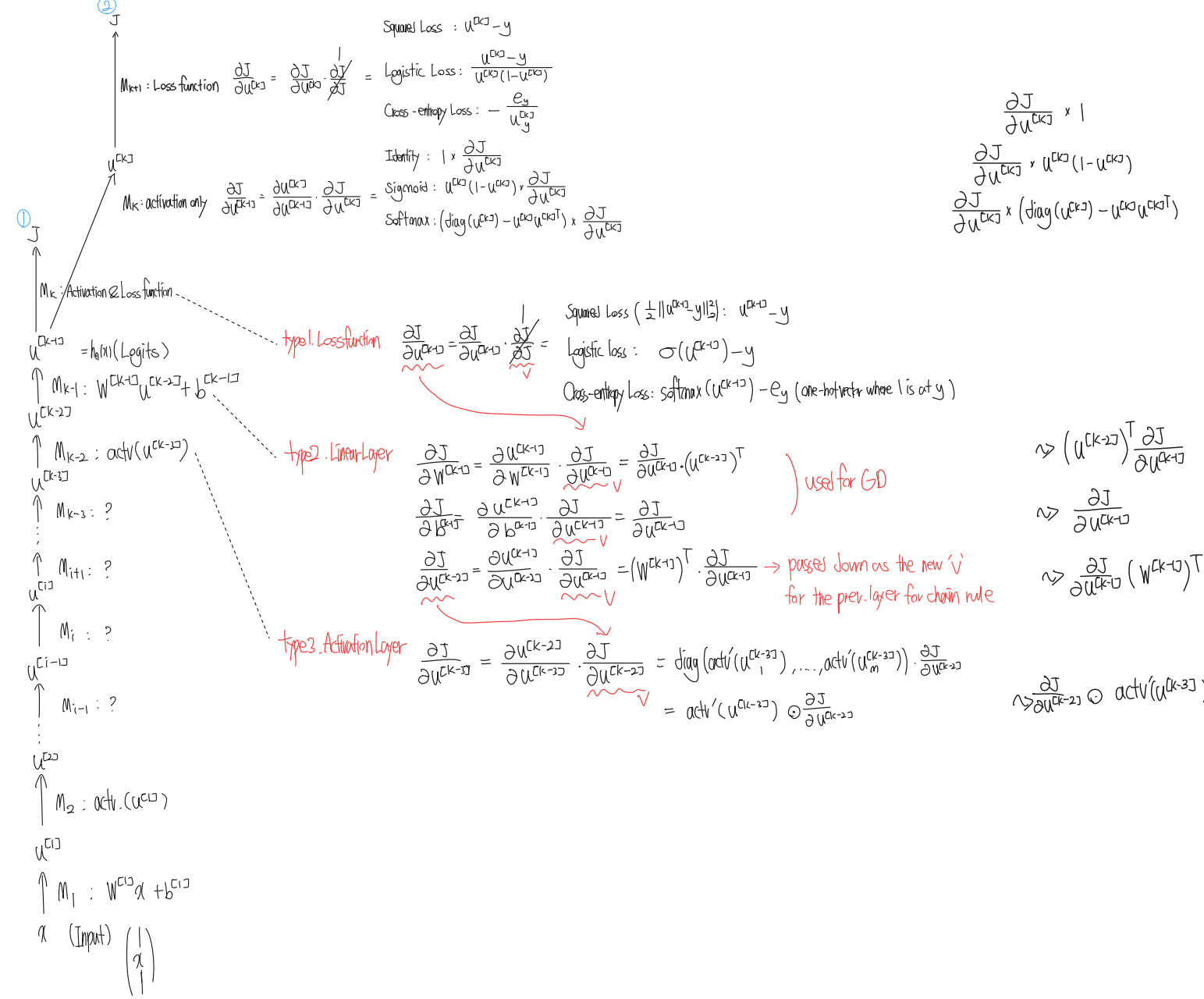


Optimization = $\min_{W^{[1]}, b^{[1]}, \dots, W^{[r]}, b^{[r]}} \frac{1}{n} \sum_{i=1}^n \ell(y^{(i)}, W^{[1]} \cdot \text{actv}(\dots \text{actv}(W^{[2]} \cdot \text{actv}(W^{[1]}(x^{(i)} + b^{[1]})) + b^{[2]})) + b^{[r]})$

Backpropagation

Forward Pass

Backward Pass



Algorithm

// Forward

$$u[0] = x$$

for $i = 1 \dots K$:

if M_i has params:

$$u[i] = M_i(u[i-1]; W_i, b_i)$$

else:

$$u[i] = M_i(u[i-1])$$

save $u[i]$

$$J = u[K]$$

// Backward

$$g[K] = 1$$

for $i = K$ down to 1:

if M_i has params:

$$dW^{(i)} = g[i] \cdot (u^{(i-1)})^T \quad // \frac{\partial J}{\partial W^{(i)}}$$

$$db^{(i)} = g[i] \quad // \frac{\partial J}{\partial b^{(i)}}$$

$$g[i-1] = (W^{(i)})^T g[i]$$

else:

$$g[i-1] = \text{act}'(u^{(i-1)}) \odot g[i]$$

// Update

for all modules i with params:

$$W^{(i)} = W^{(i)} - \eta dW^{(i)}$$

$$b^{(i)} = b^{(i)} - \eta db^{(i)}$$

↓ Vectorize over training set

// forward $\leftarrow \begin{pmatrix} x^{(1)} & x^{(2)} & \dots & x^{(n)} \end{pmatrix}$

$$U[0] = X$$

for $i = 1 \dots K$:

if M_i has params:

$$U[i] = M_i(U[i-1]; W_i, b_i)$$

else:

$$U[i] = M_i(U[i-1])$$

save $U[i]$

$$J = \text{mean}(U[K])$$

$$// U[K] = [L^{(1)}, \dots, L^{(B)}]$$

// Backward

$$G[K] = [1, 1, \dots, 1]$$

for $i = K$ down to 1:

if M_i has params:

$$dW^{(i)} = \frac{1}{B} G[i] (U^{(i-1)})^T$$

$$db^{(i)} = \frac{1}{B} G[i] \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix}$$

$$G[i-1] = (W^{(i)})^T G[i]$$

else:

$$G[i-1] = \text{act}'(U^{(i-1)}) \odot G[i]$$

// Update

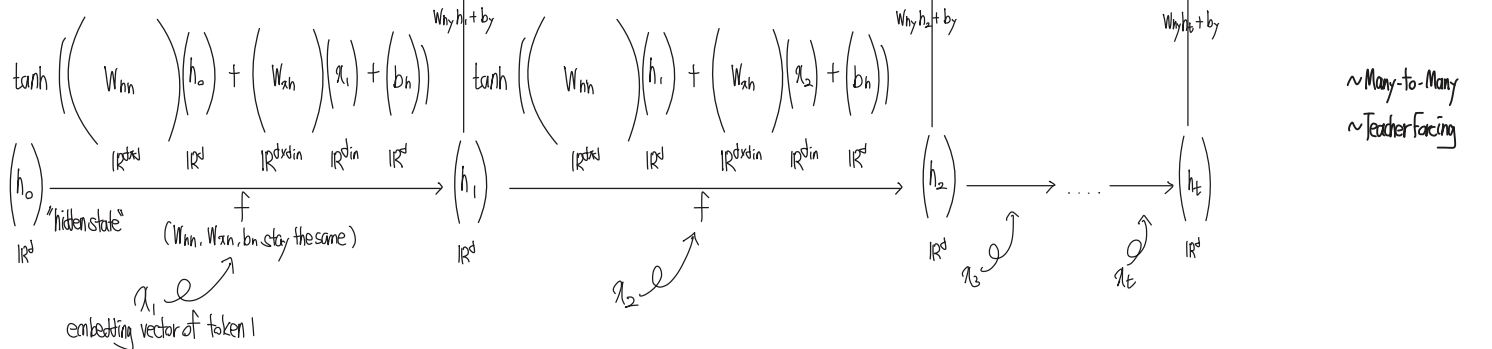
for all modules i with params:

$$W^{(i)} = W^{(i)} - \eta dW^{(i)}$$

$$b^{(i)} = b^{(i)} - \eta db^{(i)}$$

RNN (Self-Supervised Deep neural networks in time)

<Forward Pass>



- exponential problem in the forward pass, since the same params are used every time
- $\rightarrow$ tanh keeps the state bounded in $(-1, 1)^d$

<Backwards Pass>

x side ($W = W_{hh}, W_{xh}, b_h$)

$$\frac{\partial L}{\partial W} = \sum_{i=1}^t \frac{\partial L}{\partial h_i} \cdot \frac{\partial^* h_i}{\partial W} \quad \left(\frac{\partial^* h_i}{\partial W} : \text{regard } v_s \text{ inside } h_i \text{ as constants} \right)$$

$$= \text{sum} \left(\begin{aligned} &\frac{\partial L}{\partial h_t} \cdot \frac{\partial^* h_t}{\partial W} \rightarrow W_{hh} : \tanh'(W_{hh}h_{t-1} + W_{xh}x_t + b_h) \cdot h_{t-1}^T \\ &\frac{\partial L}{\partial h_t} \cdot \frac{\partial^* h_t}{\partial W} \rightarrow W_{xh} : \tanh'(W_{hh}h_{t-1} + W_{xh}x_t + b_h) \cdot x_t^T \\ &\frac{\partial L}{\partial h_t} \cdot \frac{\partial^* h_t}{\partial W} \rightarrow b_h : \tanh'(W_{hh}h_{t-1} + W_{xh}x_t + b_h) \\ &\frac{\partial L}{\partial h_{t-1}} \cdot \frac{\partial^* h_{t-1}}{\partial W} \rightarrow \frac{\partial L}{\partial h_t} = (W_{hy})^T (\text{softmax}(y_t) - e_{x_{t+1}}) \\ &\frac{\partial L}{\partial h_{t-1}} \cdot \frac{\partial^* h_{t-1}}{\partial W} \rightarrow \frac{\partial L}{\partial h_{t-1}} + \frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial h_{t-1}} + \frac{\partial L}{\partial h_t} \cdot \frac{\partial h_t}{\partial h_{t-1}} = (W_{hy})^T (\text{softmax}(y_t) - e_{x_t}) + \left(\text{diag}(\tanh'(W_{hh}h_{t-1} + W_{xh}x_t + b_h)) \cdot W_{hh} \right)^T \cdot \frac{\partial L}{\partial h_t} \\ &\frac{\partial L}{\partial h_1} \cdot \frac{\partial^* h_1}{\partial W} \rightarrow \frac{\partial L}{\partial h_1} + \frac{\partial L}{\partial h_2} + \dots + \frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial h_1} + \frac{\partial L}{\partial h_2} + \dots + \frac{\partial L}{\partial h_t} \cdot \frac{\partial h_t}{\partial h_1} = (W_{hy})^T (\text{softmax}(y_1) - e_{x_2}) + \left(\text{diag}(\tanh'(W_{hh}h_1 + W_{xh}x_2 + b_h)) \cdot W_{hh} \right)^T \cdot \frac{\partial L}{\partial h_2} \end{aligned} \right)$$

Vanishing gradient problem: W_{hh}^T is keep getting multiplied

(if > 1 : explode $\rightarrow$ can be fixed with clipping)
(if < 1 : vanish $\rightarrow$ can't really fix)

$$\frac{\partial h_t}{\partial h_k} = \prod_{k=t+1}^T \text{diag}(1 - h_k^2) W_{hh}, \quad \left\| \frac{\partial h_t}{\partial h_k} \right\| \leq \left(\prod_{k=t+1}^T \sigma_{\max}(W_{hh}) \right)^{T-k}$$

y side ($W = W_{hy} + b_y$)

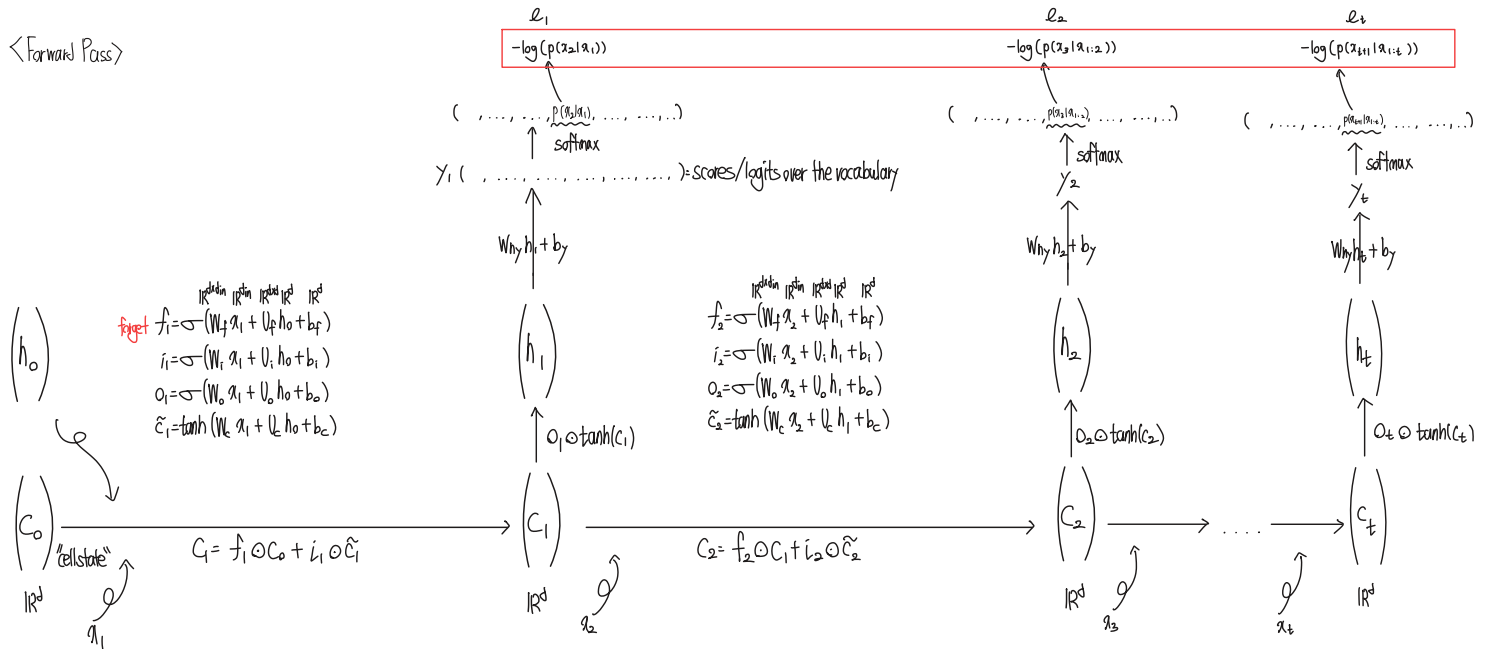
$$\frac{\partial L}{\partial W} = \sum_{i=1}^t \frac{\partial L}{\partial y_i} \cdot \frac{\partial y_i}{\partial W}$$

$$= \text{sum} \left(\begin{aligned} &\frac{\partial L}{\partial y_t} \cdot \frac{\partial y_t}{\partial W} \rightarrow W_{hy} : h_t^T \\ &\frac{\partial L}{\partial y_t} \cdot \frac{\partial y_t}{\partial W} \rightarrow b_y : 1 \\ &\frac{\partial L}{\partial y_t} \cdot \frac{\partial y_t}{\partial W} \rightarrow \frac{\partial L}{\partial y_t} = \text{softmax}(y_t) - e_{x_{t+1}} \\ &\frac{\partial L}{\partial y_1} \cdot \frac{\partial y_1}{\partial W} \rightarrow \frac{\partial L}{\partial y_1} = \text{softmax}(y_1) - e_{x_2} \end{aligned} \right)$$

LSTM // Self-Supervised

$$\text{sum} = -\log(p(a_1, a_2, \dots, a_{t+1})) \quad // \text{Loss}$$

<Forward Pass>



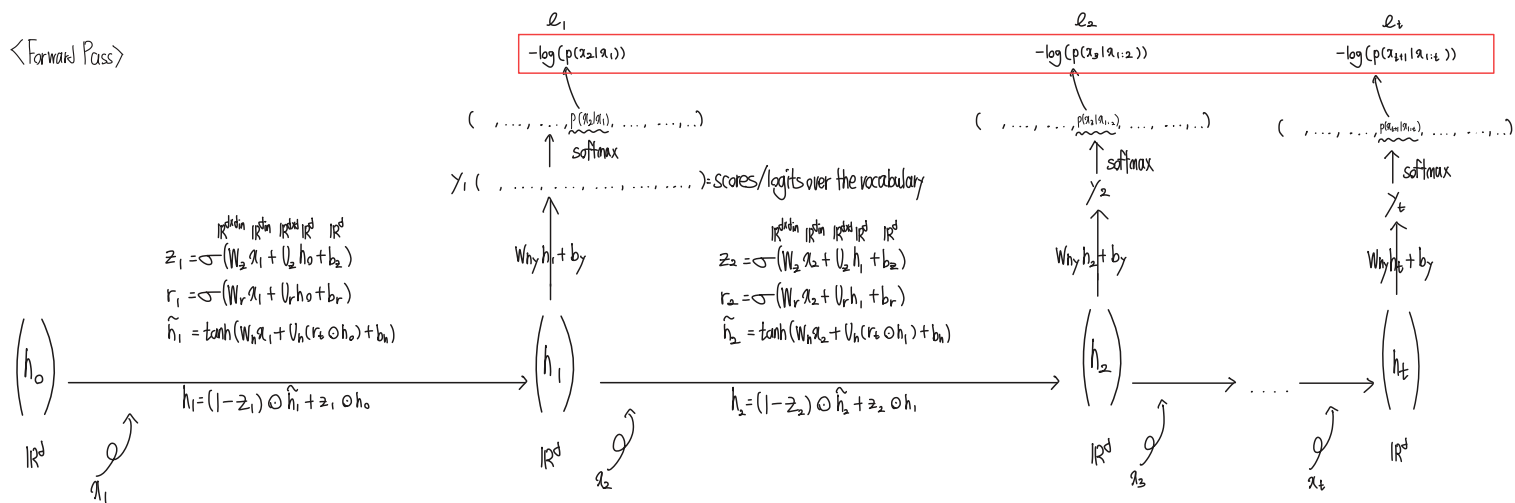
<Backward Pass> : Fixes the Vanishing gradient problem

$$\frac{\partial \mathcal{L}_t}{\partial c_t} \approx \prod_{k=t+1}^T \text{diag}(f_k)$$

GRU // Self-Supervised

$$\text{sum} = -\log(p(a_1, a_2, \dots, a_{t+1})) \quad // \text{Loss}$$

<Forward Pass>

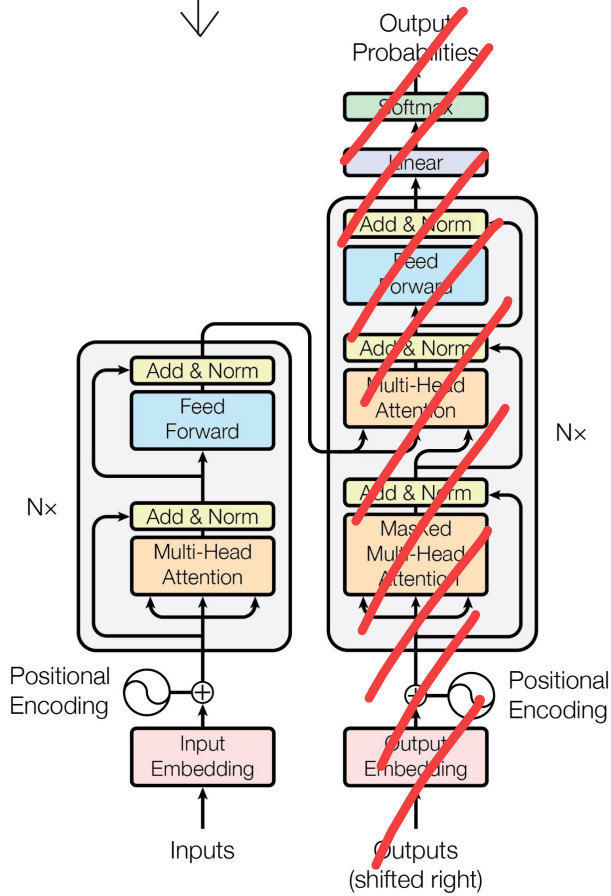
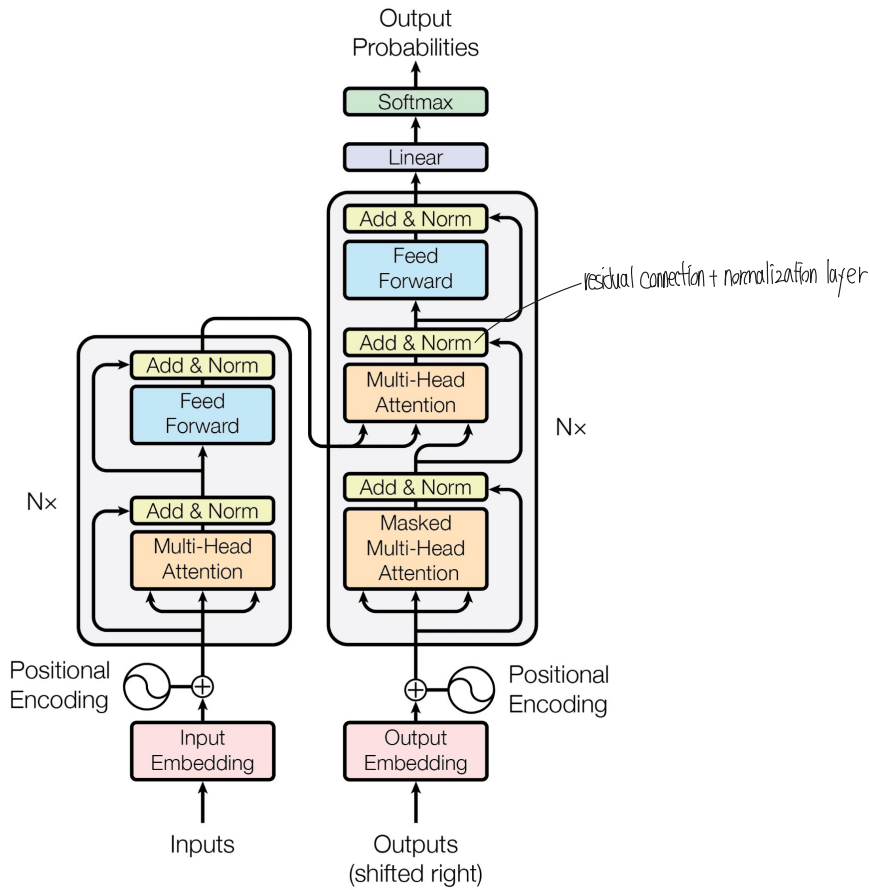


• 25% less parameters than LSTM

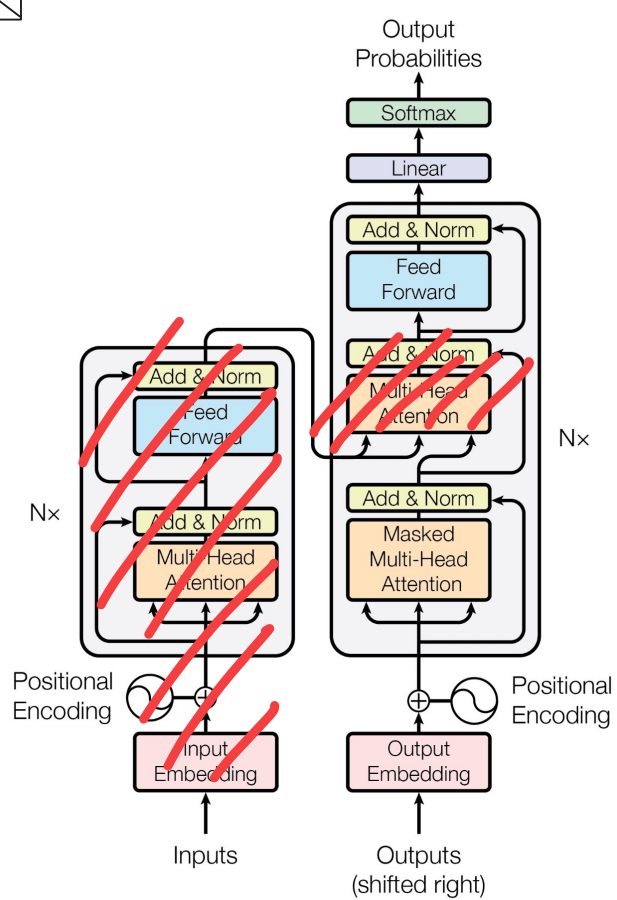
<Backward Pass> : (Also) Fixes the Vanishing gradient problem

$$\frac{\partial \mathcal{L}_t}{\partial h_t} \approx \prod_{k=t+1}^T \text{diag}(z_k)$$

Transformer $\left(\frac{\partial h^{(N)}}{\partial h^{(0)}} = \prod_{l=1}^{2N} \left(I + \frac{\partial F_l}{\partial \cdot} \right) \right)$



BERT

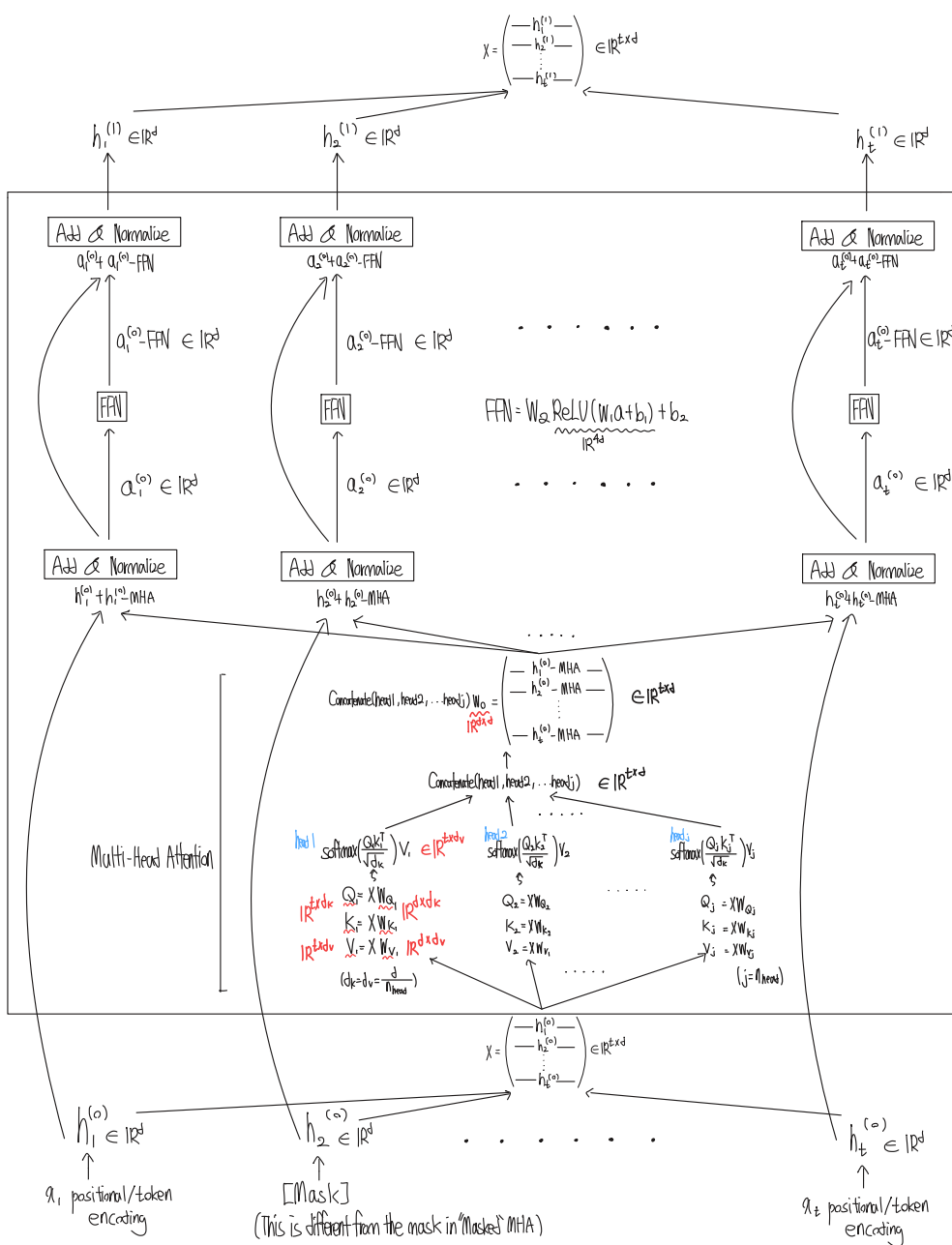
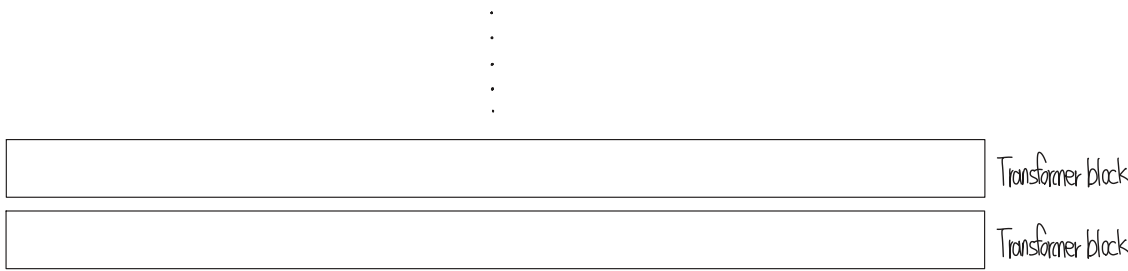
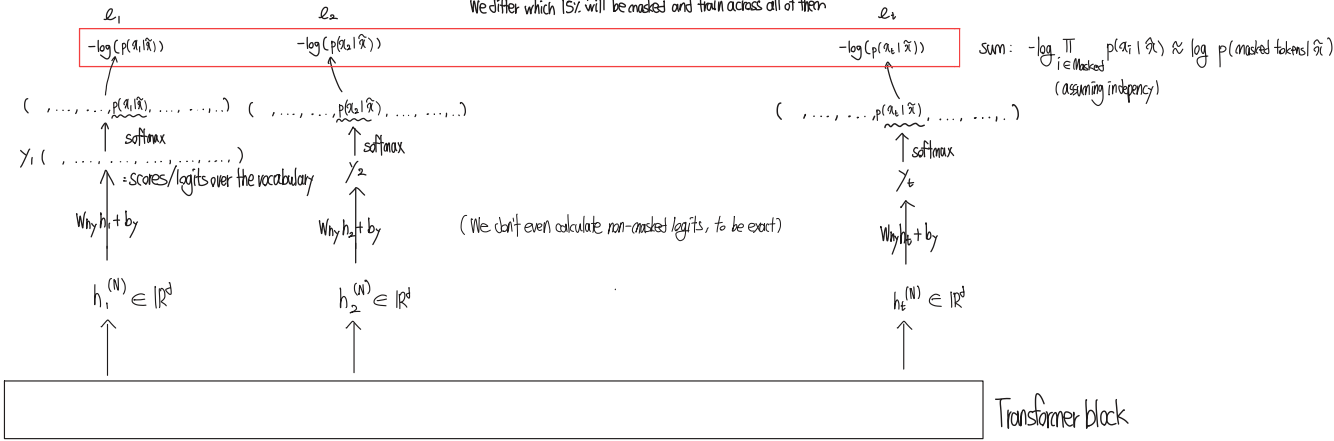


GPT

BERT // Self-Supervised

- Incapable of output, for understanding/retrieval/tagging (classification) purpose
(This is for training purposes, to perform such actual retrieval etc tasks, we tweak the architecture based on the learned parameters)

* $\tilde{x}$ is the masked sequence, 15% is masked
We differ which 15% will be masked and train across all of them



* $\frac{2}{3}$ of the params are here

* Modern: GelU, SwiGLU instead of ReLU

* MHA has t^2 in it ($\mathcal{O}(t^2)$)

So if ($t \ll d$): FFN takes more compute

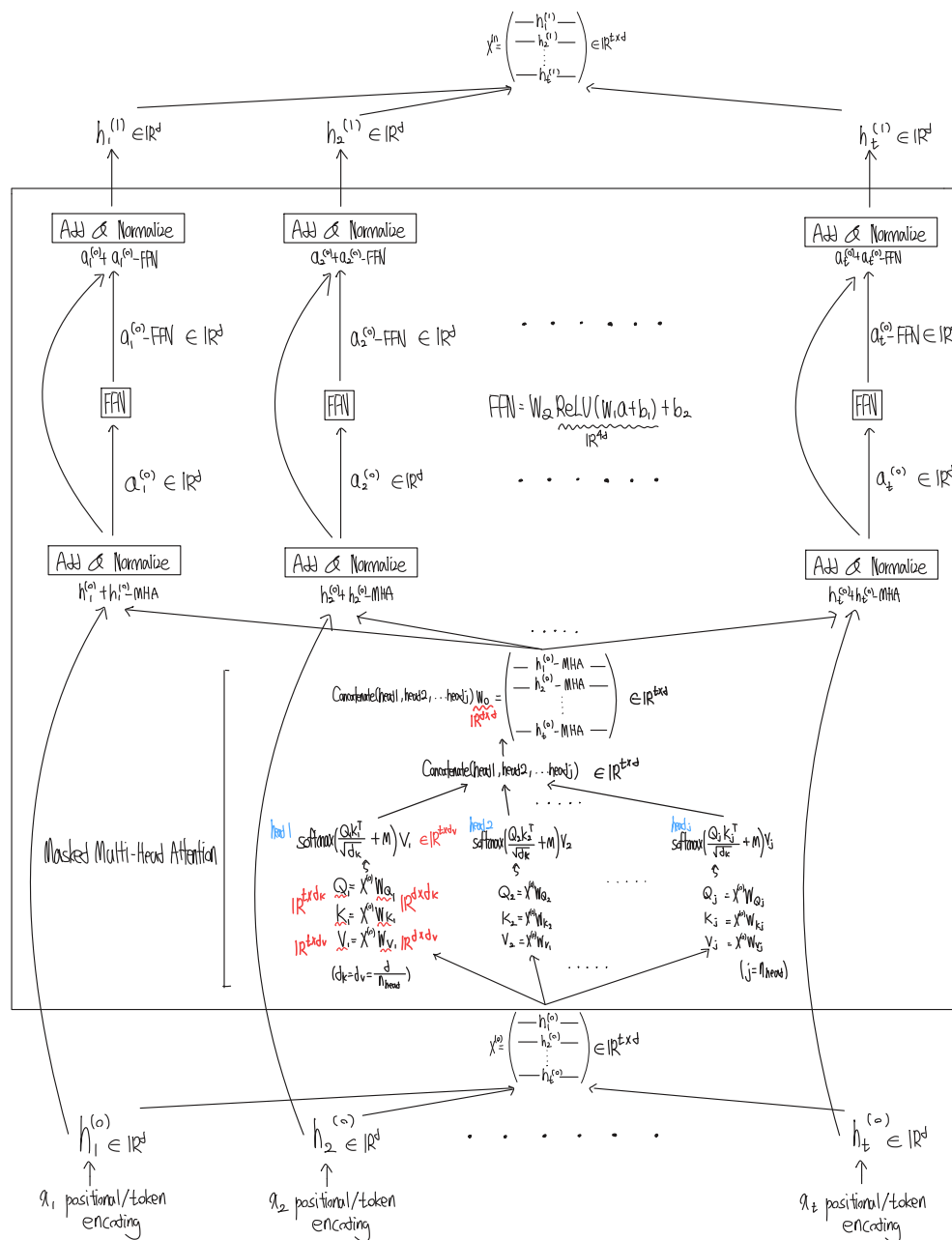
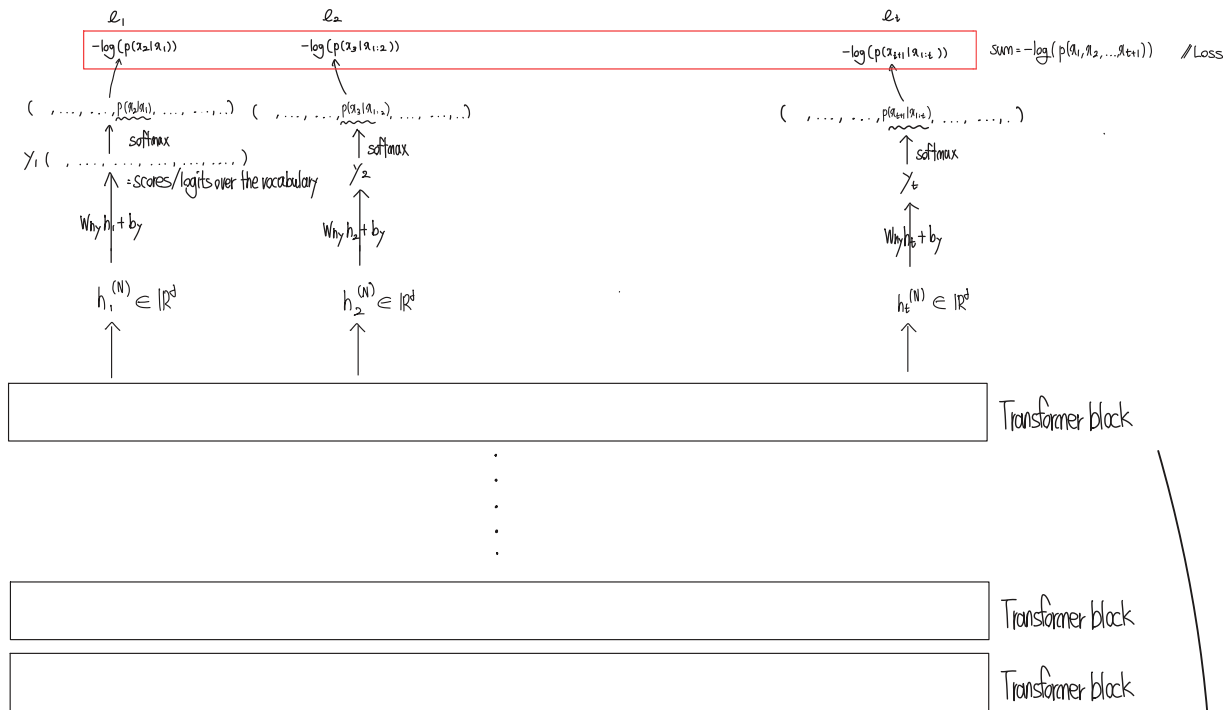
- short sentence

($t \gg d$): Attention takes more compute

- long context

Transformer block

* LayerNorm, Post-LN (Modern: Pre-LN)



N

* $\frac{2}{3}$ of the params are here

* Modern: GeLU, SwiGLU instead of ReLU

* MHA has t^2 in it ($\mathcal{O}(t^2)$)

so if ($t \ll d$): FFN takes more compute
- short sentence
($t \gg d$): Attention takes more compute
- long context

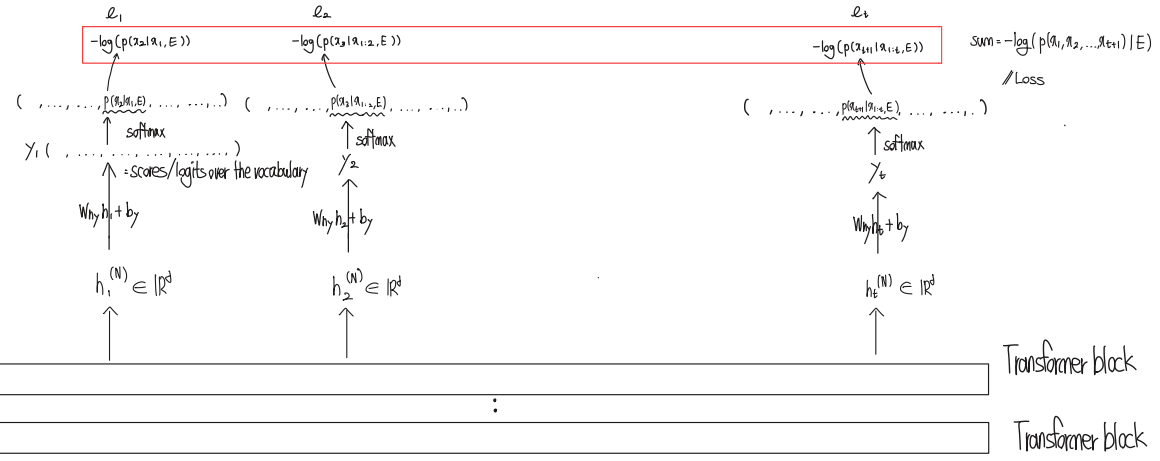
* Layernorm, Post-LN (Modern: Pre-LN)

$$* M_{ij} = \begin{cases} 0 & j \leq i \\ -\infty & j > i \end{cases}$$

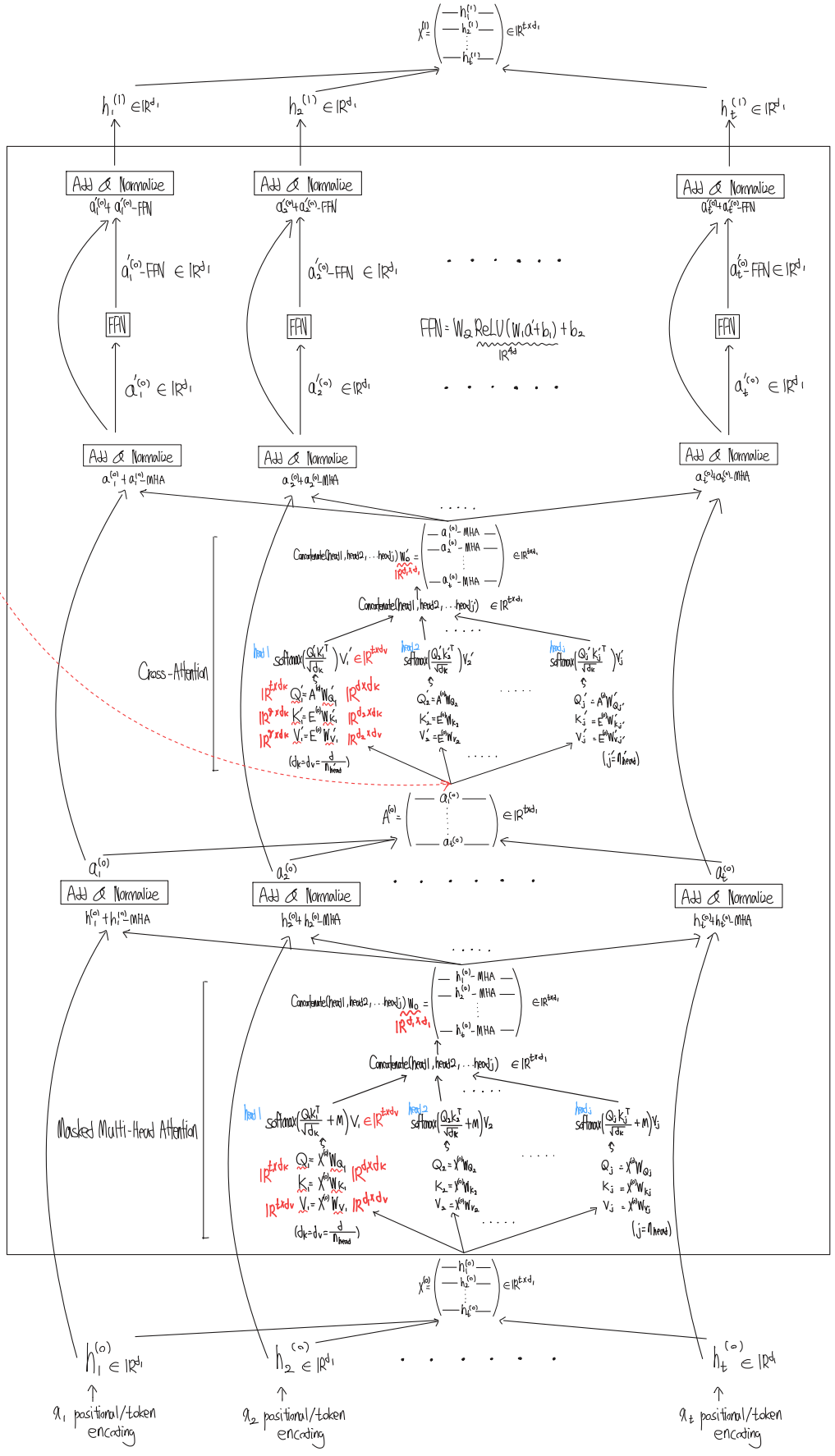
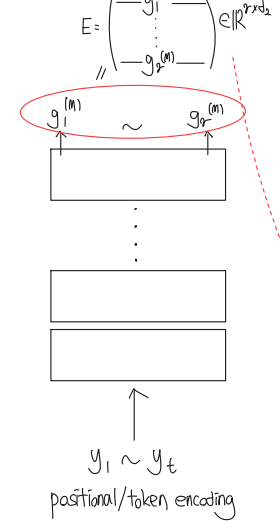
Encoder+Decoder Full

// Supervised (Discriminative)

Decoder



Encoder
(No direct loss, backpropagated through cross-attention)



Transformer block

Transformer block

Transformer Block